Supervised Learning

Slides based on Berkeley + UPenn

Reminder: HW0

- Find the most interesting example of Data Science that you can, and post the link + a short description on the forum.
- Find an awesome (publicly available) dataset, post link + short description.

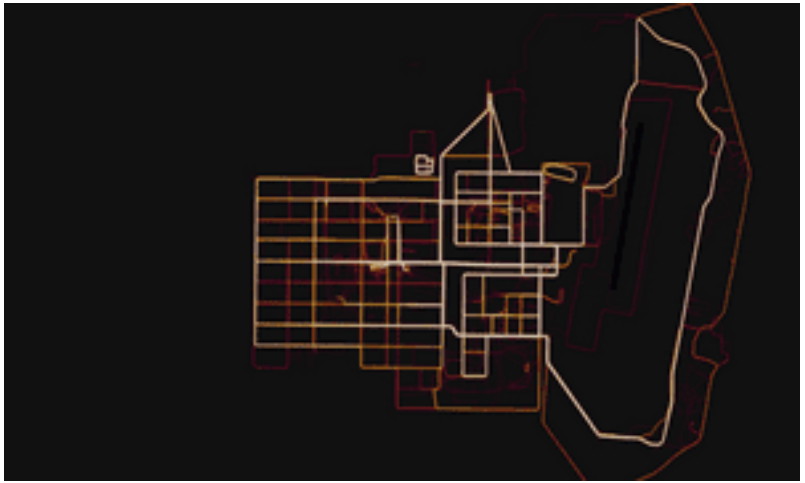
Great job :)

- Go read if you haven’t
- Some recurring themes:
  - Pop culture
    - Netflix , Spotify, movies, Steam
  - Health
    - סרטן לבלב, סוכר בדם, microscopy, AlphaFold, לחץ דם , Heart Disease, quantified self
    - מאפשר למשתמש לקבל החלטות מושכלות בזמן אמת ואף "להקדים תרופה למכה", לפני שהמשתמש חווה את התחושות (למשל עייפות) בעצמו

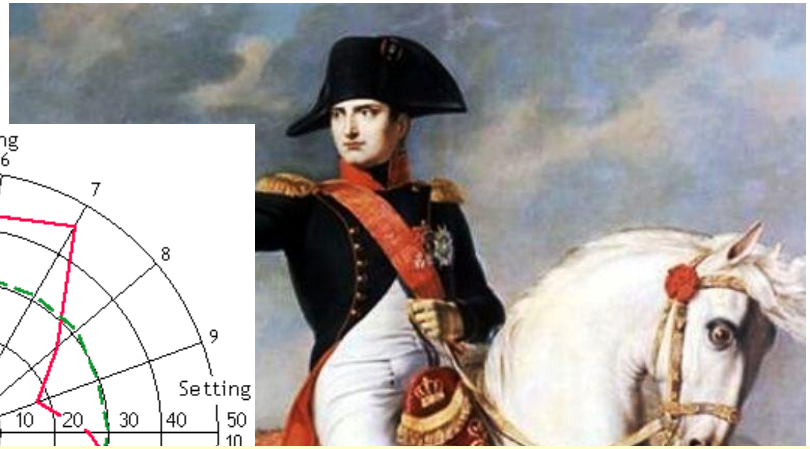
Great job :)

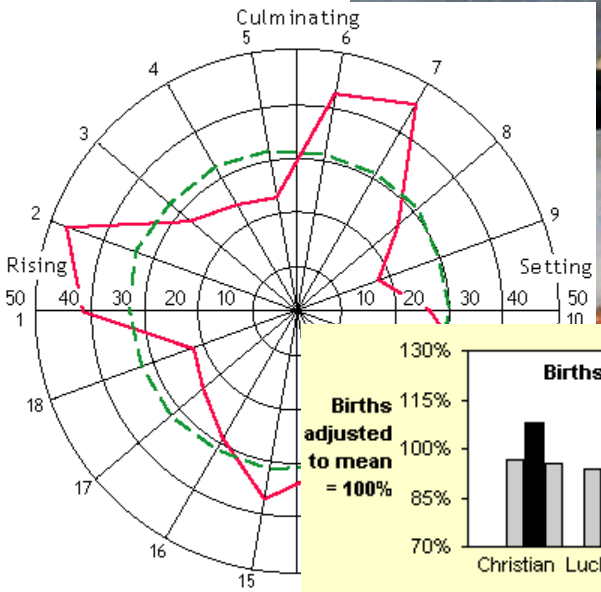
- More recurring themes:
  - Doom and gloom
    - שריפות, רעידות אדמה, fake news, rockets, Death, canIshower, wars
  - Food
    - מנות רחוב ,Random Acts of Pizza, pizzaGAN, קטשופ
  - Random
    - כלבי החלל הסובייטים ,speed dating, , הצ'ייסר, הבעות כאב בארנבונים Polymarket , UFOs, ביגפוט

Fitness tracking app Strava gives away location of secret US army bases



French Army





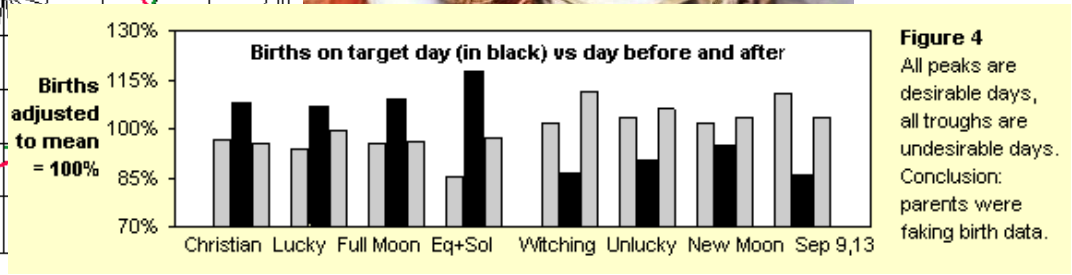
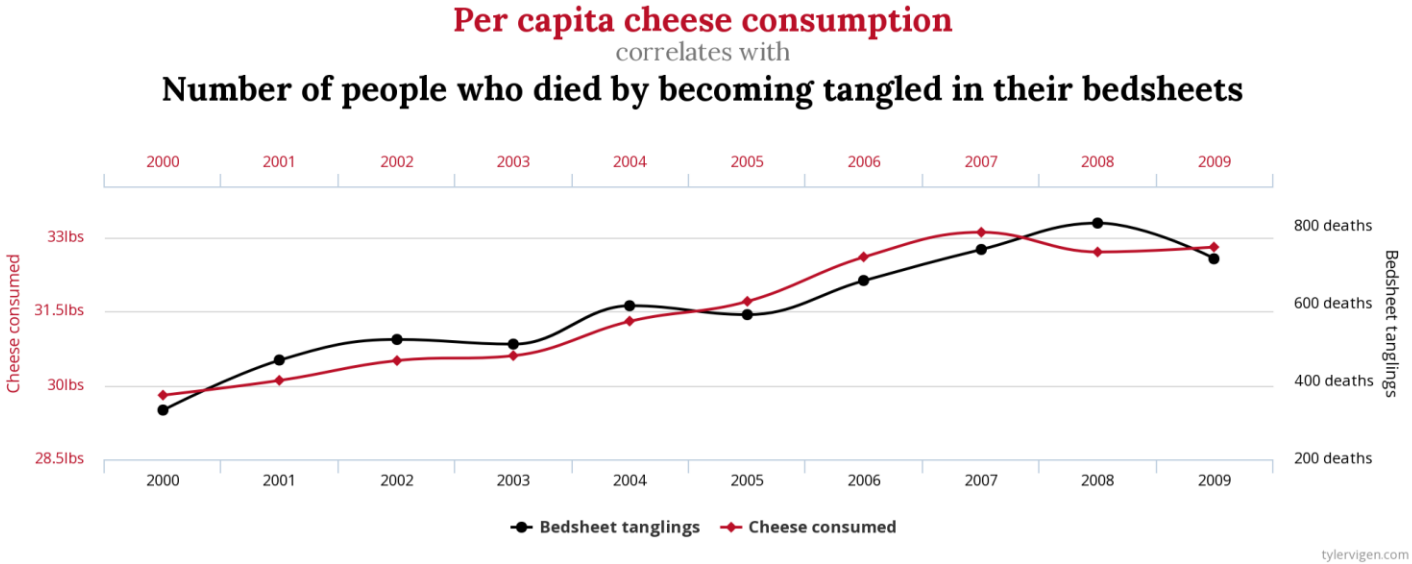


Figure 4
All peaks are desirable days, all troughs are undesirable days. Conclusion: parents were faking birth data.

Mars Effect (till 1950)

Spurious Correlations!



Projects

- Needs to be more than “we applied some ML algorithm to data” or “we visualize some correlation”
- General good ideas:
  - Have some original idea that extends/builds on what we learned in class
  - Discover interesting relationships within data
    - Combine multiple sources of data!

Examples questions from good projects

- Veganizer (using recipe data, comment data, vegan websites' data, lots of NLP, some clustering and graph algorithms)
- Revolving Doors" -- occupational shifts between public offices and leading industry executives (multiple data sources, wiki matching, graph algorithms)
- Eliezer Bot Yehuda

Examples questions from good projects

- New ways to visualize Reddit discussions (so you can easily understand what's going on in the multiple threads) – using NLP tools, sentiment analysis, discussion patterns
- A system that helps students figure out a schedule for the semester (combining recommender systems, community detection and association rules)
- Recommend stuff to Airbnb apartment owners or Yelp restaurants (if you add X you can charge more)
- Identify sleeping beauties (books that are dormant for a long time, then suddenly become popular), try to analyze why and predict the next ones

Examples of projects that weren't very ambitious

- Crawl a dataset of medications. Build a search engine that, for a given medication, returns side effects
- Find a correlation between poverty and crime rates
- Crawl sport matches, show graphs corresponding to "teams with most wins", "teams with most wins away", etc.

Improvise!

- Don’t feel limited by these
- You can collect the dataset yourself
- And define the project/question yourself

Picking Research Questions

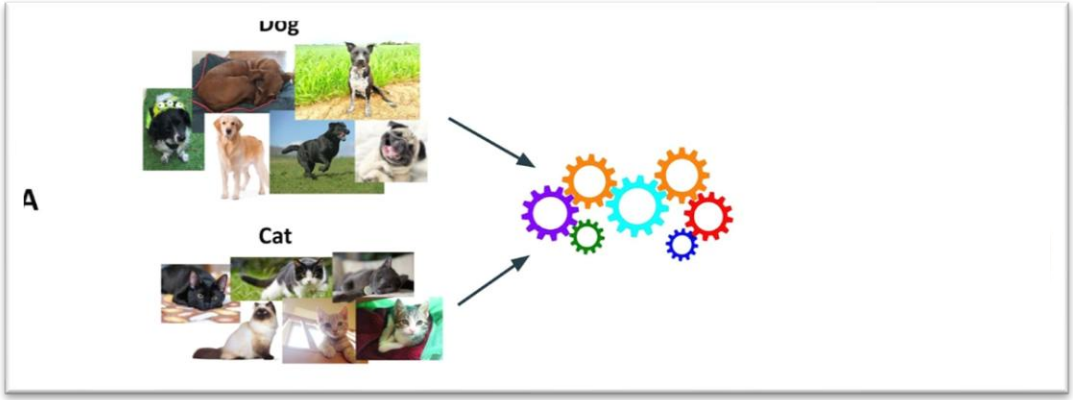
- Toy example
- Point your browsers to https://tinyurl.com/ds-huji-olympics

Supervised Learning

Slides based on Berkeley + UPenn

Supervised Learning

- Programs that learn the task from labeled examples



Supervised Learning: Framework

- We are given a set of *training examples*, consisting of *input-output* pairs (x,y), where:
  1. x is an item of the type we want to evaluate.
  2. y is the value of some function f(x).
  - Example: x is an email, and f(x) is +1 if x is spam and -1 if not.
  - Example: x is a vector giving characteristics of a voter, and y is their preferred candidate.

Framework

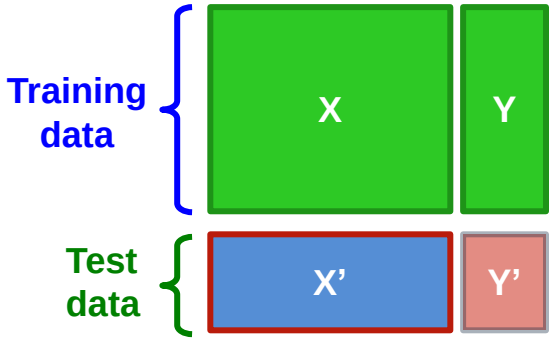
- In general, input x can be of any type.
- Often, output y is binary.
  - Usually represented as +1 = true, or “in the class” and -1 = false, or “not in the class.”
  - Called *binary classification*.
- y can be one of a finite set of categories.
  - Called *classification*.
- y can be a real number.
  - Called *regression*.
- Or really y can be of any type.

Supervised Learning

- *Supervised learning* is building a model representing the function y = f(x) from the training data.
- Example: If x and y are real numbers, the model of f might be a straight line.
- Example: if x is a email and y is +1 or -1, the model might be weights on words together with a threshold such that the answer is +1 (spam) if the weighted sum of the words in the email exceeds the threshold.
- Neural nets…

Model Training

- Divide your data randomly into training and test data.
- Build your best model based on the training data only.
- Apply your model to the test data.
- Does your model predict y' for the test data well?



The Optimization Problem

- Given **training data** and a **form of model** you wish to develop, find the instance of the model that minimizes the loss on the training data.
- Example: For the email-spam problem, incrementally adjust weights until small changes cannot decrease the probability of misclassification.

The Loss Function

- We need to decide on a *loss function* that measures how well or badly a given model represents the function y = f(x).
- Common choice: the fraction of x's for which the model gives a value different from y.
- Example: if we use a model of email spam that is a weight for each word and a threshold, then the loss for given weights + threshold = fraction of misclassified emails.

Loss Function

- If y is a numerical value, cost could be the magnitude of the difference between f(x) as computed by the model, and the true y.
  - Or its square (like RMSE).

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Loss Function

- But are all errors equally bad?
- Example: do you think that the loss should be the same for a good email classified as spam and a spam email passed to the user's inbox?

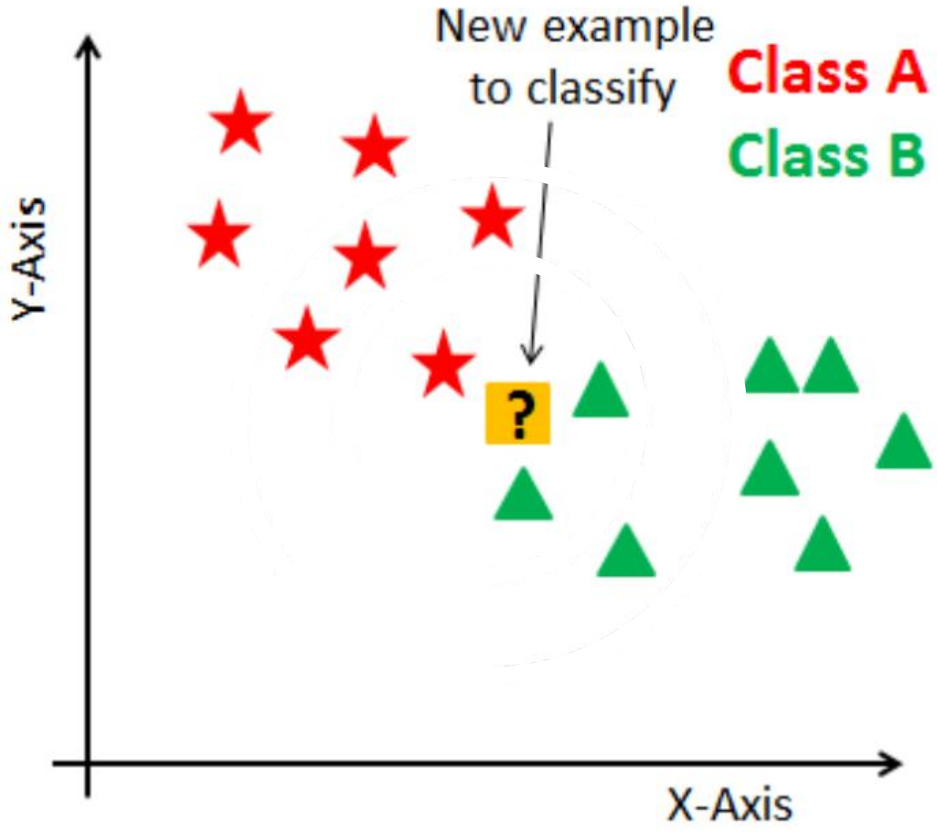
Overfitting

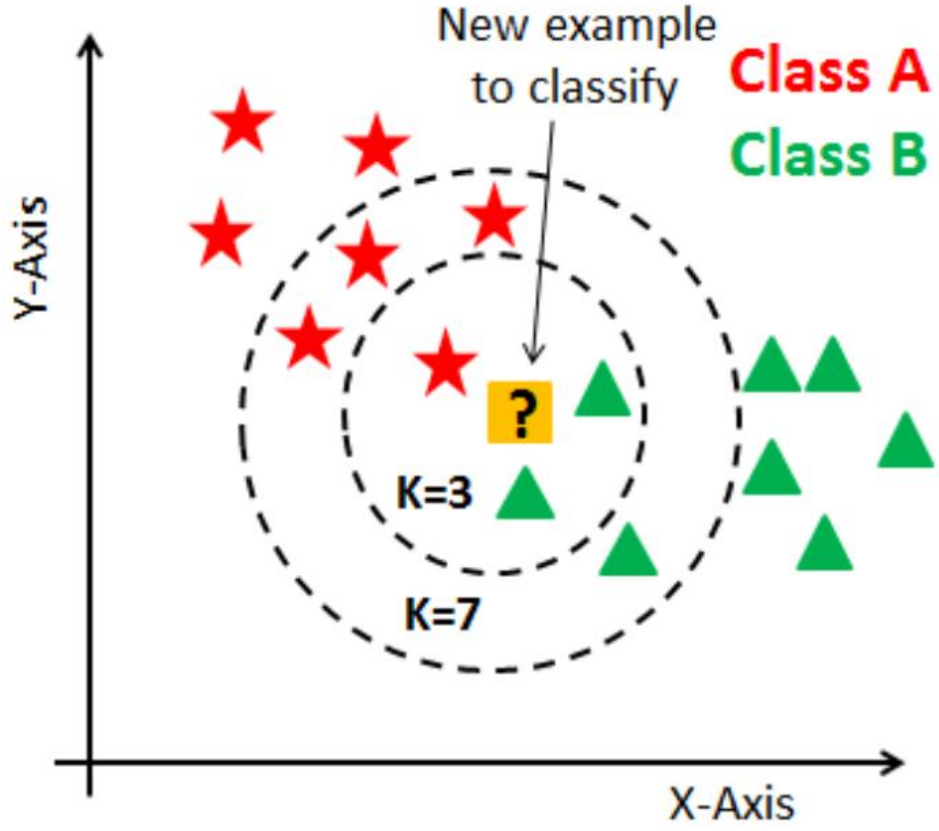
- Sometimes, your model will show much greater loss on the test data than on the training data.
  - Called *overfitting*.
- The problem is that the modeling process has picked up on details of the training data that are so fine-grained that they do not apply to the population from which the data is drawn.
- Example: a decision tree with many levels (stay tuned!)

The Real Test

- The test data helps you decide whether you are overfitting.
- But you want your model to work not only on the test data, but on all the unseen data that it will eventually see.
- If the training and test sets are truly drawn at random from the population of all data, and the test set shows little overfitting, then the model should not exhibit overfitting on real data.
  - A big “if.”

Nearest Neighbors

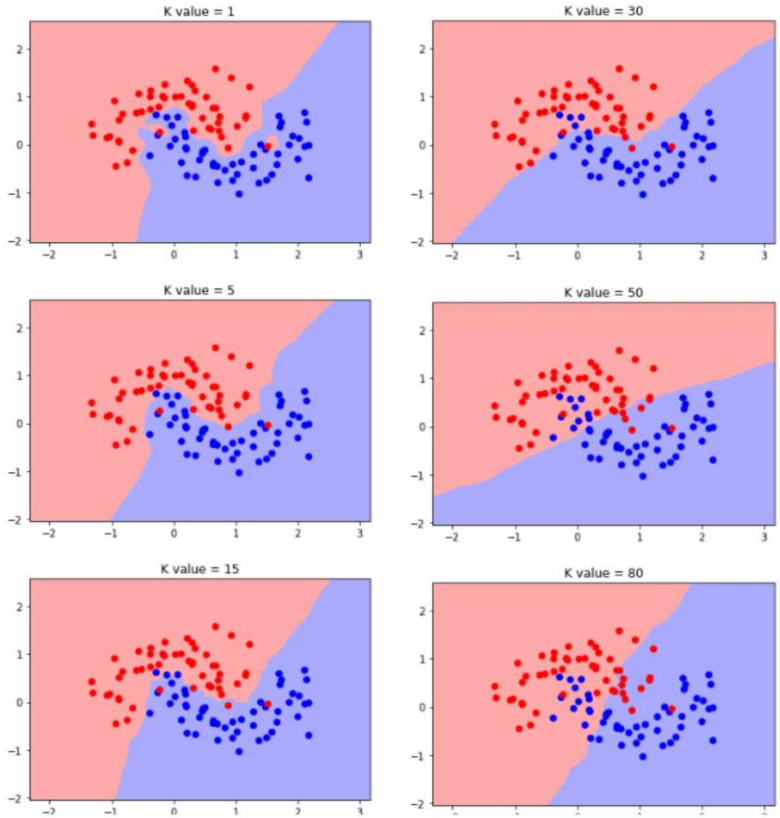

New example
to classify
Class A
Class B
Y-Axis
X-Axis
?


New example
to classify
Class A
Class B
Y-Axis
?
K=3
K=7
X-Axis

K-Nearest Neighbor (kNN) Approach

- For some chosen k, find the k training points nearest to the query point q, according to some distance measure d.
- If y is numerical, compute the average of the y's for the k nearest x's.
  - You can weight by distance or just take the average.
- If y is not numerical, combine in some appropriate way.
  - Example: If y is a category (e.g., a presidential candidate), take the value appearing most often.

Effect of k



Things to Worry About

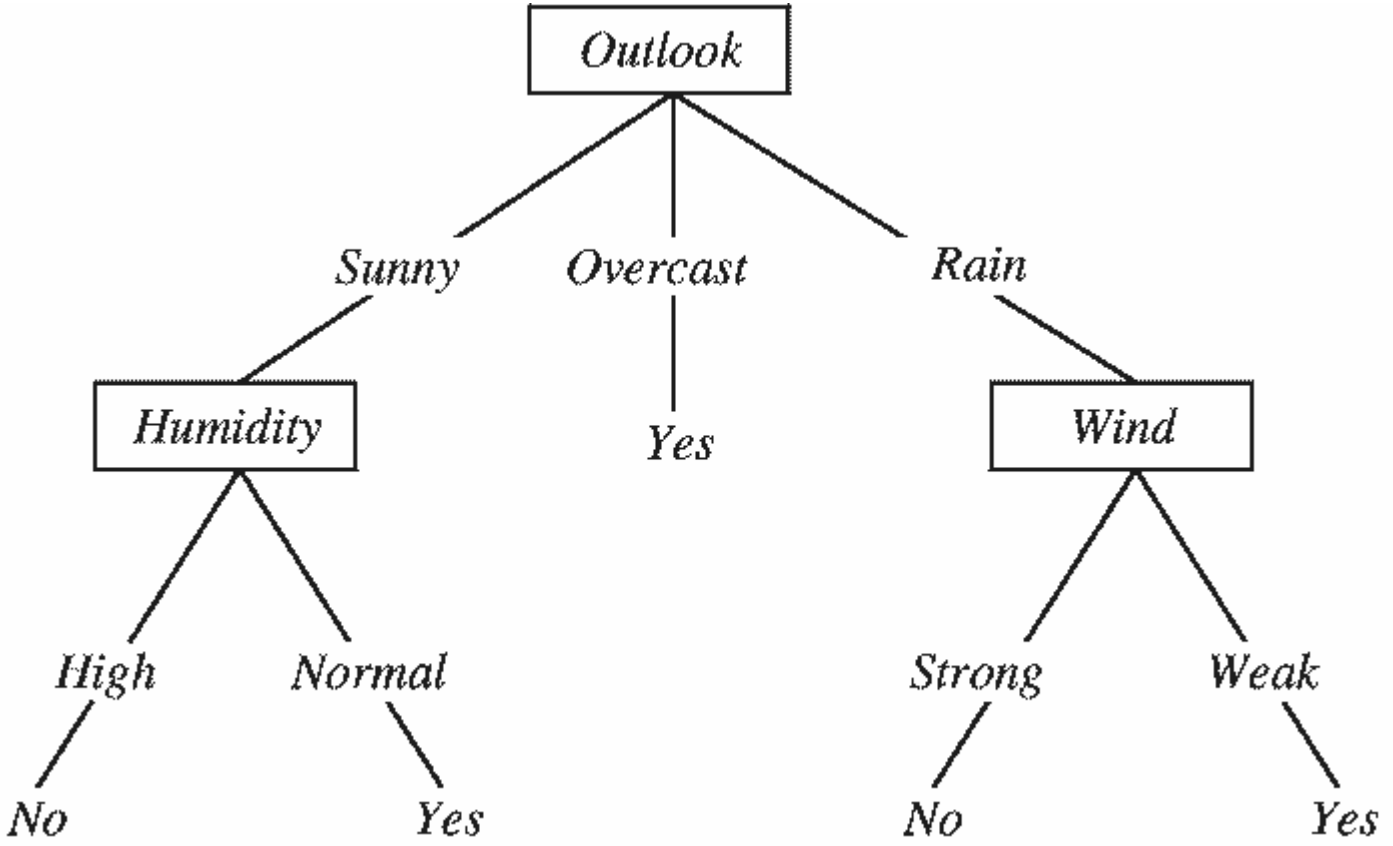
1. How do we find nearest neighbors, especially in very high-dimensional spaces?
2. How many neighbors do we consider?
3. How do we weight the influence of each near neighbor?

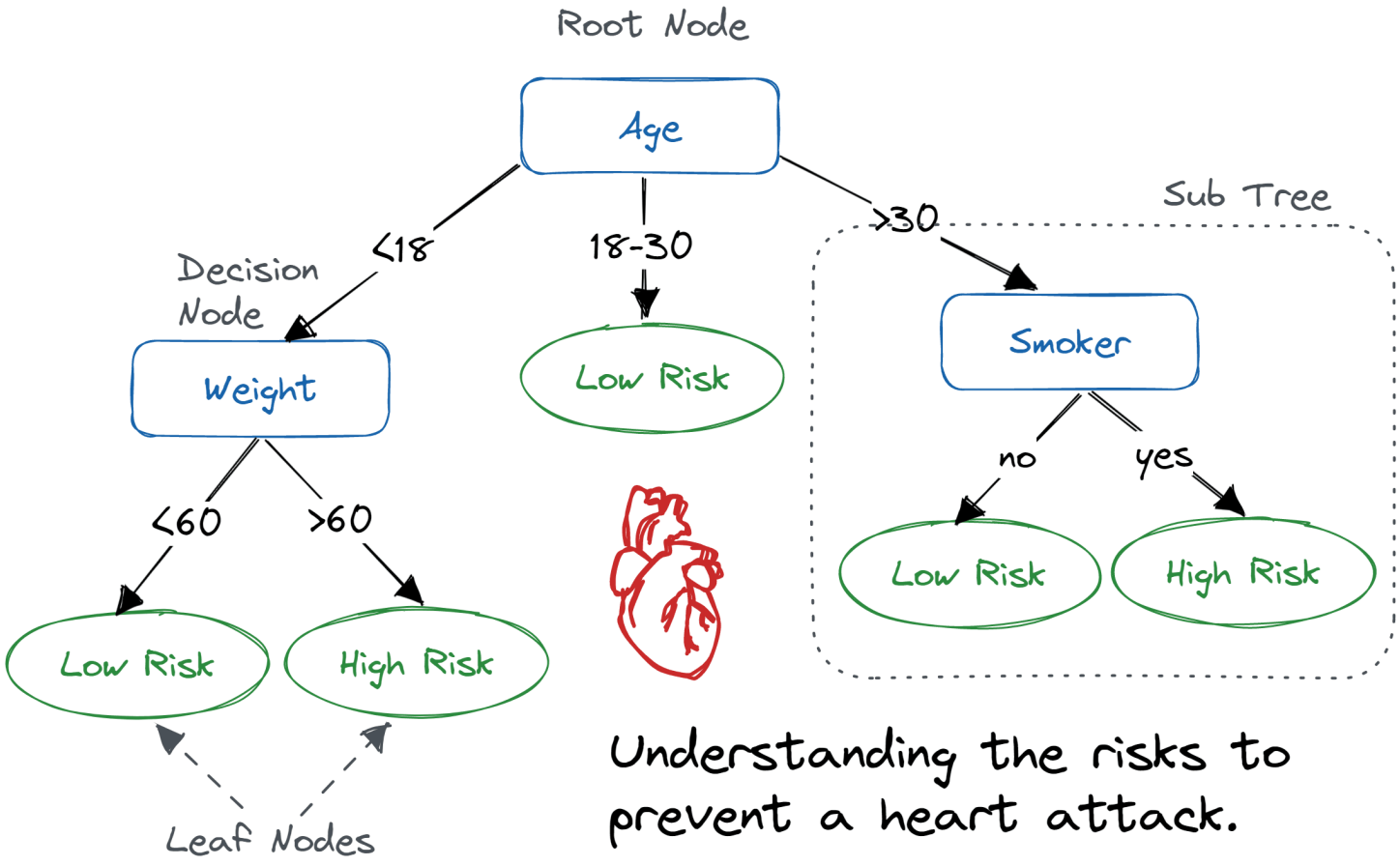
Decision Trees

Will I play tennis today?

- **Features**
  - Outlook: {Sun, Overcast, Rain}
  - Temperature: {Hot, Mild, Cool}
  - Humidity: {High, Normal, Low}
  - Wind: {Strong, Weak}

- **Labels**
  - Binary classification task: Y = {+, -}


Outlook
Sunny
Overcast
Rain
Humidity
Yes
Wind
High
Normal
Strong
Weak
No
Yes
No
Yes

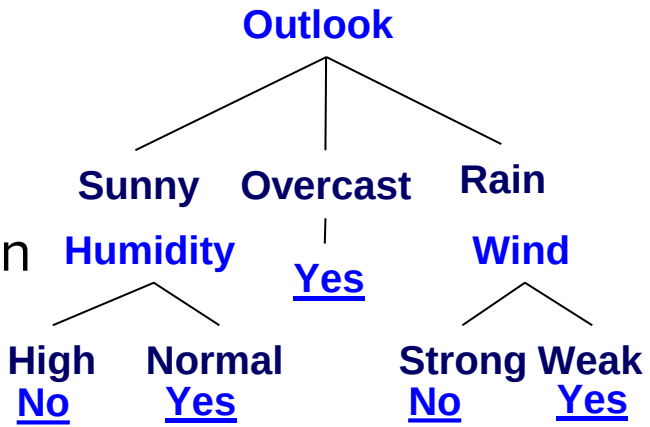


Understanding the risks to prevent a heart attack.

Decision Trees

- Can represent any Boolean Function
- Can be viewed as a way to compactly represent a lot of data.
- Natural representation: (20 questions)
- The evaluation of the Decision Tree Classifier is easy

- Clearly, given data, there are many ways to represent it as a decision tree.
- Learning a good representation from data is the challenge.



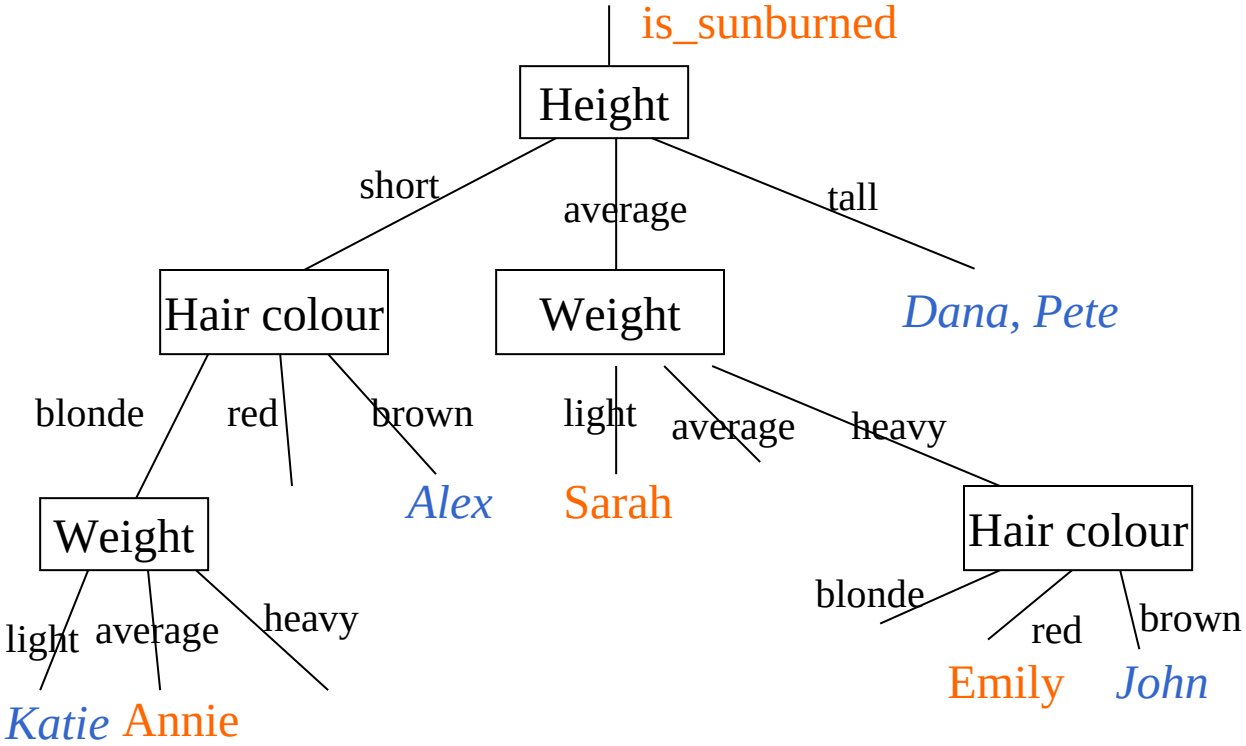
Sunburn Data Collected

Name	Hair	Height	Weight	Lotion	Result
Sarah	Blonde	Average	Light	No	Sunburned
Dana	Blonde	Tall	Average	Yes	*None*
Alex	Brown	Short	Average	Yes	*None*
Annie	Blonde	Short	Average	No	Sunburned
Emily	Red	Average	Heavy	No	Sunburned
Pete	Brown	Tall	Heavy	No	*None*
John	Brown	Average	Heavy	No	*None*
Kate	Blonde	Short	Light	Yes	*None*

Slide from A. Kaban

Decision Tree 1

correctly CLASSIFIES all the data!



Sunburn sufferers are ...

- If height="average" then
  - if weight="light" then
    - return(true) ;;; Sarah
  - elseif weight="heavy" then
    - if hair_colour="red" then
      - return(true) ;;; Emily
- elseif height="short" then
  - if hair_colour="blonde" then
    - if weight="average" then
      - return(true) ;;; Annie
- else *return(false) ;;;everyone else*

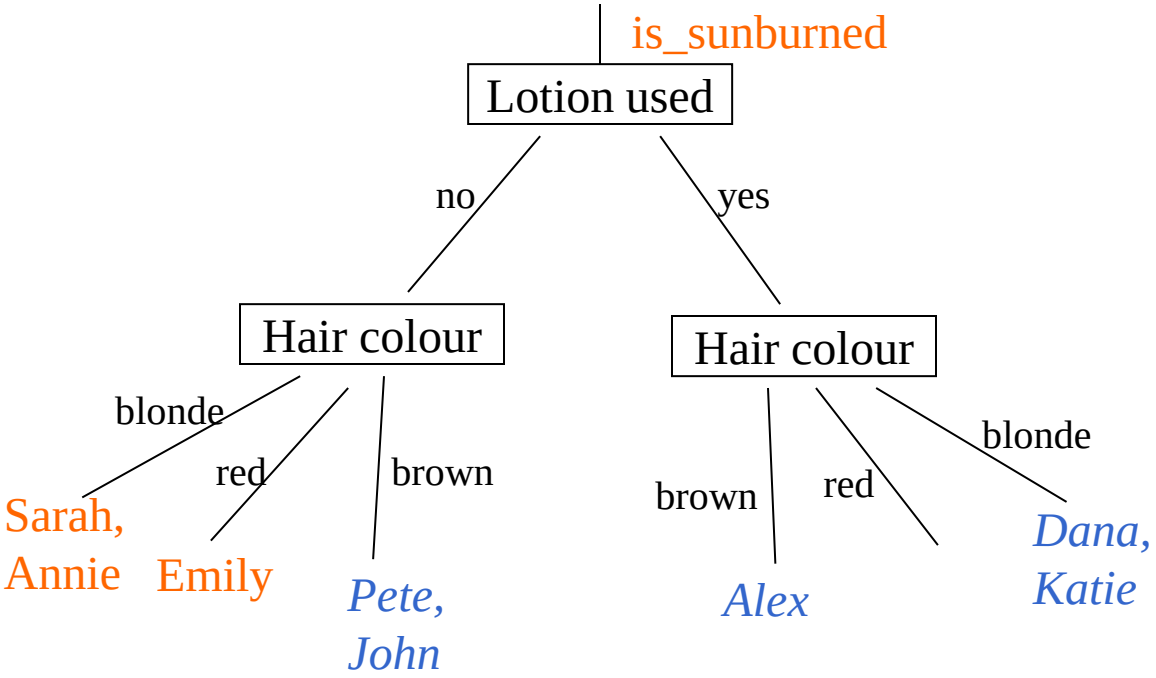
Slide from A. Kaban

Can you do better?

Name	Hair	Height	Weight	Lotion	Result
Sarah	Blonde	Average	Light	No	Sunburned
Dana	Blonde	Tall	Average	Yes	*None*
Alex	Brown	Short	Average	Yes	*None*
Annie	Blonde	Short	Average	No	Sunburned
Emily	Red	Average	Heavy	No	Sunburned
Pete	Brown	Tall	Heavy	No	*None*
John	Brown	Average	Heavy	No	*None*
Kate	Blonde	Short	Light	Yes	*None*

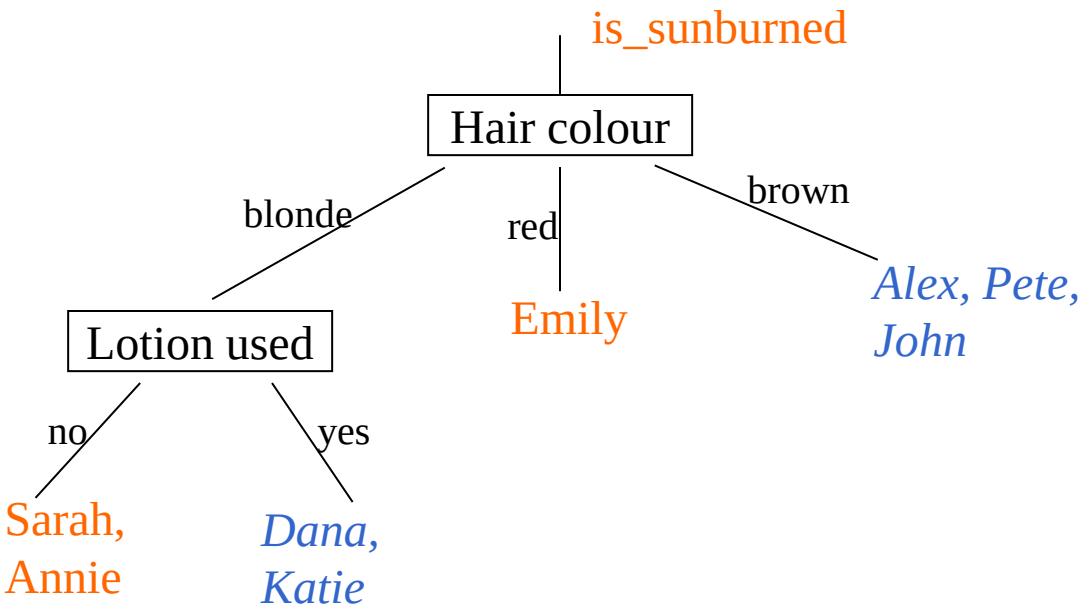
Slide from A. Kaban

Decision Tree 2



Slide from A. Kaban

Decision Tree 3



Slide from A. Kaban

Summing up

- Irrelevant attributes do not classify the data well
- Using irrelevant attributes thus causes larger decision trees
- a computer could look for simpler decision trees
- Q: How?

 Slide from A. Kaban

Basic Decision Tree Algorithm

- Let S be the set of Examples
  - Label is the target attribute (the prediction)
  - Attributes is the set of measured attributes
- ID3(*S*, Attributes, Label)

If all examples are labeled the same return a single node tree with Label

Otherwise Begin

A = attribute in Attributes that *best* classifies S (Create a Root node for tree)

for each possible value v of A

Add a new tree branch corresponding to A=v

Let *Sv* be the subset of examples in S with A=v

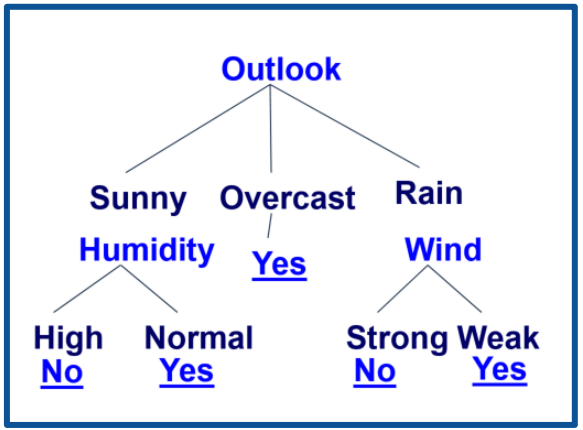
if *Sv* is empty: add leaf node with the common value of Label in S

Else: below this branch add the subtree

ID3(*Sv*, Attributes - {a}, Label)

End

Return Root



Picking the Root Attribute

- The goal is to have the resulting decision tree as small as possible (Occam's Razor)
  - But, finding the minimal decision tree consistent with the data is NP-hard
- The recursive algorithm is a greedy heuristic search for a simple tree, but cannot guarantee optimality.
- The main decision in the algorithm is the selection of the next attribute to condition on.

Picking the Root Attribute

- Consider data with two Boolean attributes (A,B).

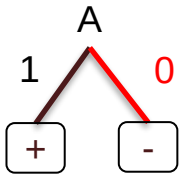
  < (A=0,B=0), - >: 50 examples

  < (A=0,B=1), - >: 50 examples

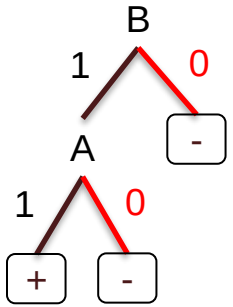
  < (A=1,B=0), - >: 0 examples

  < (A=1,B=1), + >: 100 examples

- What should be the first attribute we select?
  - Splitting on A: we get purely labeled nodes.
  - Splitting on B: we don't get purely labeled nodes.



splitting on A



splitting on B

Picking the Root Attribute

- We want attributes that split the examples to sets that are relatively pure in one label; this way we are closer to a leaf node.
  - The most popular heuristics is based on information gain, originated with the ID3 system of Quinlan.

Entropy

- Entropy (impurity, disorder) of a set of examples, S, relative to a binary classification is:

$$Entropy(S) = -p_+ \log(p_+) - p_- \log(p_-)$$

- p_+ is the proportion of positive examples in S and
- p_- is the proportion of negatives examples in S
  - If all the examples belong to the same category: Entropy = 0
  - If all the examples are equally mixed (0.5, 0.5): Entropy = 1
  - Entropy = Level of uncertainty.
- In general, when p_i is the fraction of examples labeled i:

$$Entropy(S[p_1, p_2, \ldots, p_k]) = -\sum_1^k p_i \log(p_i)$$

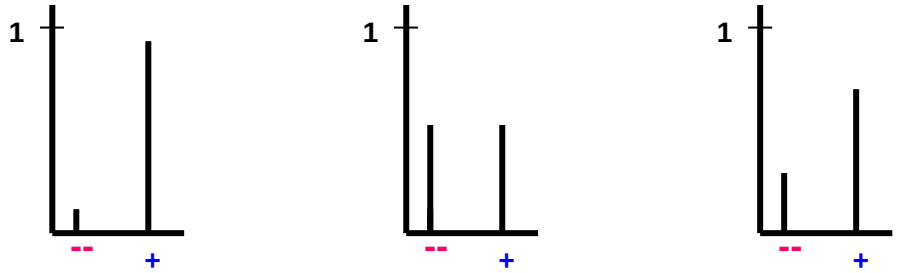
- Entropy can be viewed as the number of bits required, on average, to encode the class of labels. If the probability for + is 0.5, a single bit is required for each example; if it is 0.8 – can use less then 1 bit.

Entropy

- Entropy (impurity, disorder) of a set of examples, S, relative to a binary classification is:

$$Entropy(S) = -p_+ \log(p_+) - p_- \log(p_-)$$

- p_+ is the proportion of positive examples in S and
- p_- is the proportion of negatives examples in S
  - If all the examples belong to the same category: Entropy = 0
  - If all the examples are equally mixed (0.5, 0.5): Entropy = 1
  - Entropy = Level of uncertainty.



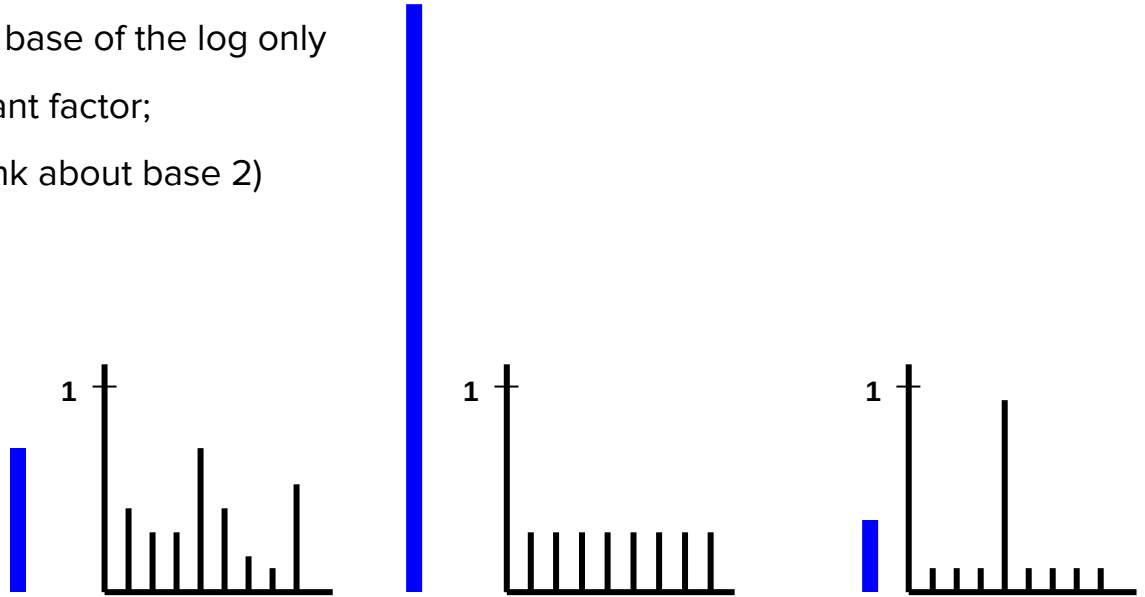
Entropy

$$Entropy(S[p_1, p_2\ , \ldots, p_k]) = -\sum_1^k p_i \log(p_i)$$

(Convince yourself that the max value would be $\log(k)$)

(Also note that the base of the log only introduce a constant factor;

therefore, we'll think about base 2)



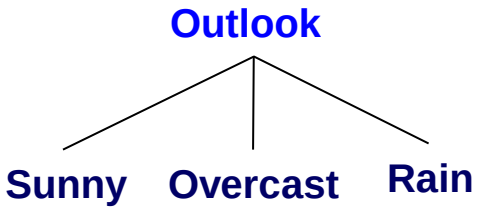
Information Gain

High Entropy – High level of Uncertainty

Low Entropy – No Uncertainty.

- The information gain of an attribute a is the expected reduction in entropy caused by partitioning on this attribute

$$Gain(S, a) = Entropy(S) - \sum_{v \in values(S)} \frac{|S_v|}{|S|} Entropy(S_v)$$



- Where:
  - S_v is the subset of S for which attribute a has value v, and
  - the entropy of partitioning the data is calculated by **weighing the entropy of each partition** by its size relative to the original set

- Partitions of low entropy lead to high gain
- Go back to check which of the A, B splits is better

Will I play tennis today?

	O	T	H	W	Play?
1	S	H	H	W	-
2	S	H	H	S	-
3	O	H	H	W	+
4	R	M	H	W	+
5	R	C	N	W	+
6	R	C	N	S	-
7	O	C	N	S	+
8	S	M	H	W	-
9	S	C	N	W	+
10	R	M	N	W	+
11	S	M	N	S	+
12	O	M	H	S	+
13	O	H	N	W	+
14	R	M	H	S	-

Outlook: S(unny), O(vercast), R(ainy)

Temperature: H(ot), M(edium), C(ool)

Humidity: H(igh), N(ormal), L(ow)

Wind: S(trong), W(eak)

Will I play tennis today?

	O	T	H	W	Play?
1	S	H	H	W	-
2	S	H	H	S	-
3	O	H	H	W	+
4	R	M	H	W	+
5	R	C	N	W	+
6	R	C	N	S	-
7	O	C	N	S	+
8	S	M	H	W	-
9	S	C	N	W	+
10	R	M	N	W	+
11	S	M	N	S	+
12	O	M	H	S	+
13	O	H	N	W	+
14	R	M	H	S	-

calculate current entropy

- $p_+ = \frac{9}{14} \quad p_- = \frac{5}{14}$
- $Entropy(Play) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$

$$= -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14}$$

$$\approx 0.94$$

Information Gain: Outlook

	O	T	H	W	Play?
1	S	H	H	W	-
2	S	H	H	S	-
3	O	H	H	W	+
4	R	M	H	W	+
5	R	C	N	W	+
6	R	C	N	S	-
7	O	C	N	S	+
8	S	M	H	W	-
9	S	C	N	W	+
10	R	M	N	W	+
11	S	M	N	S	+
12	O	M	H	S	+
13	O	H	N	W	+
14	R	M	H	S	-

$$Gain(S,a) = Entropy(S) - \sum_{v \in values(S)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Outlook = sunny:

$p_+ = 2/5 \quad p_- = 3/5$ **Entropy(O = S) = 0.971**

Outlook = overcast:

$p_+ = 4/4 \quad p_- = 0$ **Entropy(O = O) = 0**

Outlook = rainy:

$p_+ = 3/5 \quad p_- = 2/5$ **Entropy(O = R) = 0.971**

Expected entropy

= $\sum_{v \in values(S)} \frac{|S_v|}{|S|} Entropy(S_v)$

= (5/14)×0.971 + (4/14)×0 + (5/14)×0.971 = **0.694**

Information gain = 0.940 – 0.694 **= 0.246**

Information Gain: Humidity

	O	T	H	W	Play?
1	S	H	H	W	-
2	S	H	H	S	-
3	O	H	H	W	+
4	R	M	H	W	+
5	R	C	N	W	+
6	R	C	N	S	-
7	O	C	N	S	+
8	S	M	H	W	-
9	S	C	N	W	+
10	R	M	N	W	+
11	S	M	N	S	+
12	O	M	H	S	+
13	O	H	N	W	+
14	R	M	H	S	-

$$Gain(S,a) = Entropy(S) - \sum_{v \in values(S)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Humidity = high:

$p_+ = 3/7 \quad p_- = 4/7$ **Entropy(H = H) = 0.985**

Humidity = Normal:

$p_+ = 6/7 \quad p_- = 1/7$ **Entropy(H = N) = 0.592**

Expected entropy

= $\sum_{v \in values(S)} \frac{|S_v|}{|S|} Entropy(S_v)$

= (7/14)×0.985 + (7/14)×0.592 = **0.7885**

Information gain = 0.940 – 0.7785 **= 0.1515**

Which feature to split on?

	O	T	H	W	Play?
1	S	H	H	W	-
2	S	H	H	S	-
3	O	H	H	W	+
4	R	M	H	W	+
5	R	C	N	W	+
6	R	C	N	S	-
7	O	C	N	S	+
8	S	M	H	W	-
9	S	C	N	W	+
10	R	M	N	W	+
11	S	M	N	S	+
12	O	M	H	S	+
13	O	H	N	W	+
14	R	M	H	S	-

Information gain:

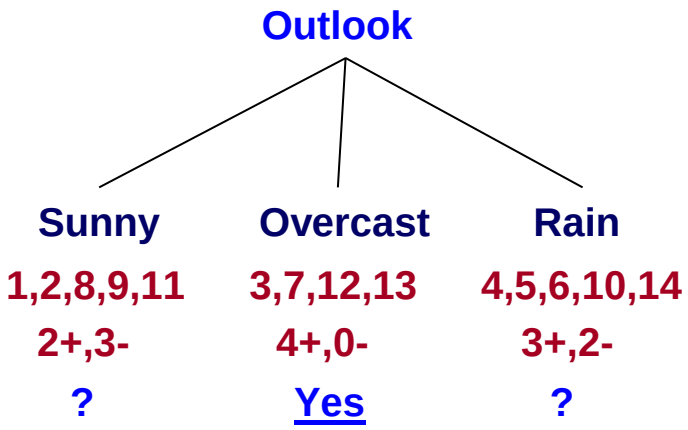
Outlook: 0.246
Humidity: 0.151
Wind: 0.048
Temperature: 0.029

→ Split on Outlook

An Illustrative Example (III)

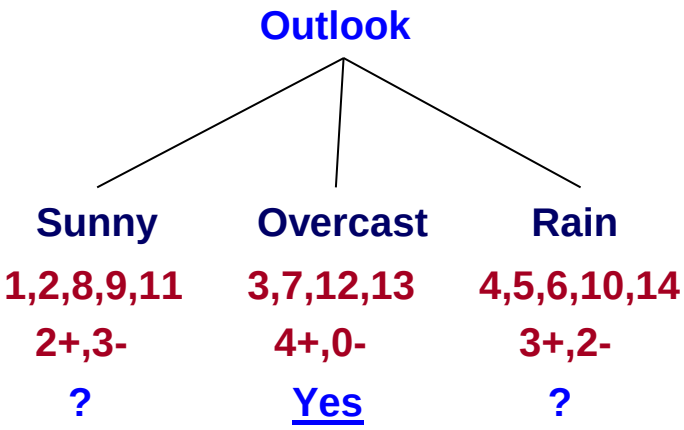
Gain(S,Humidity)=0.151
Gain(S,Wind) = 0.048
Gain(S,Temperature) = 0.029
Gain(S,Outlook) = 0.246

An Illustrative Example (III)



	O	T	H	W	Play?
1	S	H	H	W	-
2	S	H	H	S	-
3	O	H	H	W	+
4	R	M	H	W	+
5	R	C	N	W	+
6	R	C	N	S	-
7	O	C	N	S	+
8	S	M	H	W	-
9	S	C	N	W	+
10	R	M	N	W	+
11	S	M	N	S	+
12	O	M	H	S	+
13	O	H	N	W	+
14	R	M	H	S	-

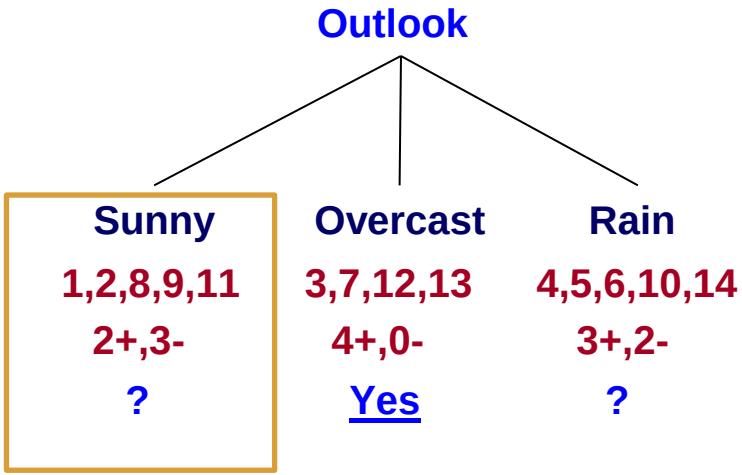
An Illustrative Example (III)



Continue until:
- Every attribute is included in path, or,
- All examples in the leaf have same label

	O	T	H	W	Play?
1	S	H	H	W	-
2	S	H	H	S	-
3	O	H	H	W	+
4	R	M	H	W	+
5	R	C	N	W	+
6	R	C	N	S	-
7	O	C	N	S	+
8	S	M	H	W	-
9	S	C	N	W	+
10	R	M	N	W	+
11	S	M	N	S	+
12	O	M	H	S	+
13	O	H	N	W	+

An Illustrative Example (IV)



$Gain(S_{sunny}, Humidity) = .97-(3/5)\ 0-(2/5)\ 0 = .97$

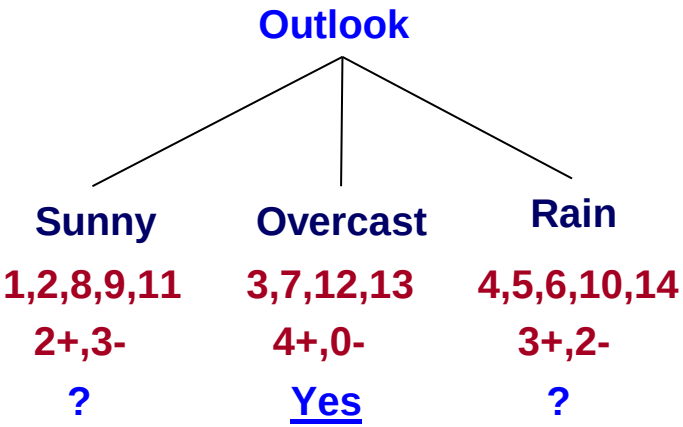
$Gain(S_{sunny}, Temp) = .97-\ 0-(2/5)\ 1 = .57$

$Gain(S_{sunny}, Wind) = .97-(2/5)\ 1 - (3/5)\ .92 = .02$

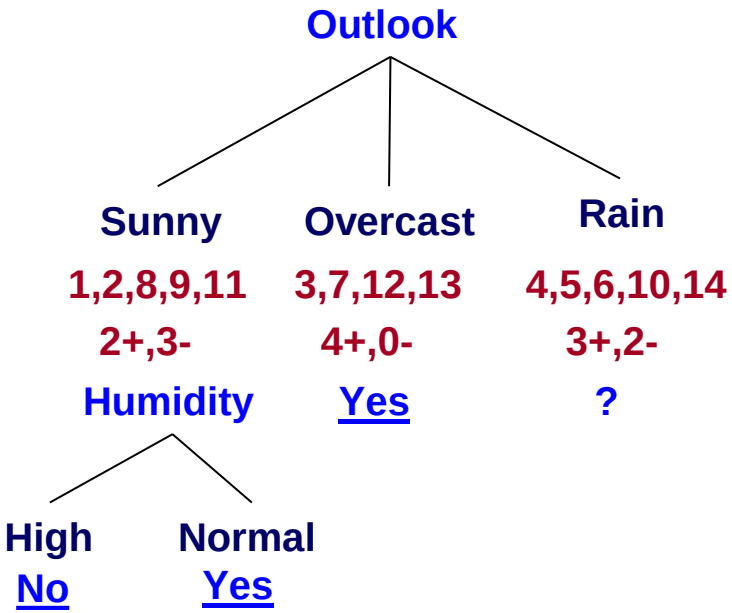
Split on Humidity

	O	T	H	W	Play?
1	S	H	H	W	-
2	S	H	H	S	-
4	R	M	H	W	+
5	R	C	N	W	+
6	R	C	N	S	-
7	O	C	N	S	+
8	S	M	H	W	-
9	S	C	N	W	+
10	R	M	N	W	+
11	S	M	N	S	+
12	O	M	H	S	+
13	O	H	N	W	+
1	R	M	H	S	-

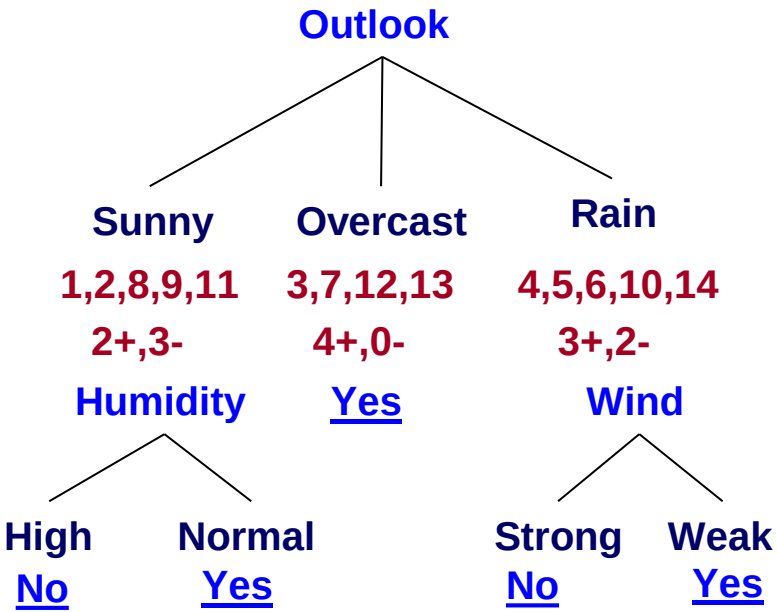
An Illustrative Example (V)



An Illustrative Example (V)



An Illustrative Example (VI)



Avoiding Overfitting

- Learning a tree that classifies the training data perfectly may not lead to the tree with the best generalization performance.
  - There may be noise in the training data the tree is fitting
  - The algorithm might be making decisions based on very little data
- Two basic approaches
  - Pre-pruning: Stop growing the tree at some point during construction when it is determined that there is not enough data to make reliable choices.
  - Post-pruning: Grow the full tree and then remove nodes that seem not to have sufficient evidence.